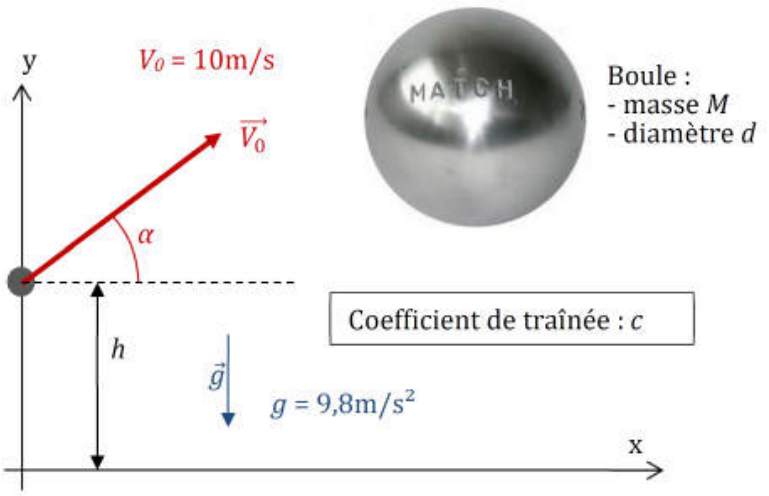
# Résolution d'équation $f(x) = 0$
# TIR DE PÉTANQUE

## 1 Présentation du problème

On considère dans cette application l'étude de trajectoire d'une boule de pétanque. On considère ici un tir tendu à 30° comme défini sur la figure ci-dessous.







Boule :
- masse $M$
- diamètre $d$



**Données :**
$M$ = 0,7 kg
$d$ = 70mm
$c$ = 0,47
$\alpha = \pi/6$
$h$ = 1,5m

Les frottements de l'air sont pris en compte et génèrent une force de frottement proportionnelle au carré de la vitesse :

$$F = \frac{1}{2}\rho c S V^2$$

Cette force est alors fonction du coefficient de traînée $c$, de la masse volumique de l'air $\rho$ que l'on prendra égale à 1,2 kg.m$^{-3}$ (conditions à 22°C et 1013 hPa), de la surface projetée de la boule $S$.

Dans la suite on notera $k = \frac{1}{2}\rho c S$.

Le principe fondamental de la dynamique appliquée à la boule de pétanque donne :

$$M\vec{a} = M\vec{g} - \vec{F}$$

Après résolution, on peut en déduire les 2 systèmes d'équations horaires pour la position sur chaque axe :

$$x(t) = \frac{M}{k} \ln\left(\frac{kV_0 t\cos\alpha + M}{M}\right)$$

$$y(t) = h + \frac{M}{k} \ln\left(\cos\left(\sqrt{\frac{gk}{M}}t - \arctan\left(\sqrt{\frac{gk}{M}}V_0\sin\alpha\right)\right)\right) - \frac{M}{2k}\ln\left(\frac{Mg}{k.(V_0\sin\alpha)^2 + Mg}\right)$$

Si on néglige les forces de frottements de l'air sur la boule, on aboutit aux équations simplifiées suivantes :

$$x(t) = V_0 t\cos\alpha$$

$$y(t) = h - \frac{1}{2}gt^2 + V_0 t\sin\alpha$$

## 2 Travail demandé

**Activité 1**

- À partir des données numériques fournies, déterminer en utilisant la méthode de Newton (`newton2`), la position $x_0$ du point d'impact sur le sol dans le cas où l'on néglige la force de frottement de l'air sur la boule.

```
import math as m

# Définition des valeurs numériques de l'étude :
V0=10
g=9.81
h=1.5
c=0.47
M=0.7
a=m.pi/6
d=0.07
S=((m.pi)*d**2)/4
rho=1.2
k=(1/2)*rho*c*S

#---------------ETUDE SANS FROTTEMENTS---------------------------------

# Définition des équations horaires :

def vx(t):
    vx=V0*m.cos(a)
    return vx

def vy(t):
    vy=-g*t+V0*m.sin(a)
    return vy

def x(t):
    x=V0*m.cos(a)*t
    return x

```

```python
def y(t):
    y=h-(1/2)*g*t**2+V0*m.sin(a)*t
    return y

#---------------RESOLUTION y(t)=0-------------------

# On utilise la méthode "newton2" pour ne pas avoir à calculer la dérivée

print("\n**************************************************************************")
print("Résolution SANS prise en compte des forces de frottement dans l'air")
print("**************************************************************************")

from Newton import newton2

tol=0.000001; # A microseconde près
maxiter=100; # 100 itérations au maximum
ti=2

t0=newton2(y,ti,tol,maxiter)
x0=x(t0)
print("\nLe point d'impact est à {} mètres du tireur".format(x0))
```

**Activité 2**

- Comparer cette valeur dans le cas où les forces de frottement de l'air ne sont pas négligées.

```python
#---------------ETUDE AVEC FROTTEMENTS-----------------------------------

# Définition des équations horaires :

def vxf(t):
    vx=(M*V0*m.cos(a))/(k*V0*m.cos(a)*t+M)
    return vx

def vyf(t):
    arctgte=m.atan(m.sqrt(k/(M*g))*V0*m.sin(a))
    vy=m.sqrt(M*g/k)*m.tan(t*m.sqrt(k/(M*g))-arctgte)
    return vy

def xf(t):
    x=(M/k)*m.log((k*V0*m.cos(a)*t+M)/M)
    return x

def yf(t):
    arctgte=m.atan(V0*m.sin(a)*m.sqrt(k/(M*g)))
    A=m.cos(t*m.sqrt(g*k/M)-arctgte)
    B=M*g/(k*(V0*m.sin(a))**2 + M*g)
    y=h + (M/k)*m.log(A) - (M/(2*k))*m.log(B)
    return y

#---------------RESOLUTION y(t)=0-------------------

# On utilise la méthode "newton2" pour ne pas avoir à calculer la dérivée

print("\n**************************************************************************")
```

```
print("Résolution AVEC prise en compte des forces de frottement dans l'air")
print("*****************************************************************")

from Newton import newton2

tol=0.000001; # A 1 microseconde près
maxiter=100; # 100 itérations au maximum
ti=2

tf0=newton2(yf,ti,tol,maxiter)
xf0=xf(tf0)
print("\nLe point d'impact est à {} mètres du tireur".format(xf0))
```

**Activité 3**

- Tracer les trajectoires de la boule dans le plan $(\overrightarrow{x}, \overrightarrow{y})$, pour chaque modélisation (avec et sans prise en compte des frottements de l'air sur la boule).

```
#--------------TRACES DES COURBES---------------------
import matplotlib.pyplot as plt

plt.close("all")

t=[]; xx=[]; yy=[];
tf=[]; xxf=[]; yyf=[];
for i in range(101):
    t.append(t0*i/100)
    xx.append(x(t[i]))
    yy.append(y(t[i]))
    tf.append(tf0*i/100)
    xxf.append(xf(t[i]))
    yyf.append(yf(t[i]))
figure1 = plt.figure()
plt.title("Trajectoire boule de pétanque")
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.plot(xx,yy,color='b')
plt.plot(xxf,yyf,color='r')
plt.axis('equal')
plt.show()
```

**Activité 4**

- Reprendre cette étude en considérant une balle de tennis de masse $M_t = 56\,\text{g}$.

```
import math as m

# Définition des valeurs numériques de l'étude :
V0=10
```

```
g=9.81
h=1.5
c=0.47
M=0.056
a=m.pi/6
d=0.07
S=((m.pi)*d**2)/4
rho=1.2
k=(1/2)*rho*c*S


#----------------ETUDE SANS FROTTEMENTS----------------------------------

# Définition des équations horaires :

def vx(t):
    vx=V0*m.cos(a)
    return vx

def vy(t):
    vy=-g*t+V0*m.sin(a)
    return vy

def x(t):
    x=V0*m.cos(a)*t
    return x

def y(t):
    y=h-(1/2)*g*t**2+V0*m.sin(a)*t
    return y

#---------------RESOLUTION y(t)=0-------------------

# On utilise la méthode "newton2" pour ne pas avoir à calculer la dérivée

print("\n**********************************************************************")
print("Résolution SANS prise en compte des forces de frottement dans l'air")
print("**********************************************************************")

from Newton import newton2

tol=0.000001; # A microseconde près
maxiter=100; # 100 iterations au maximum
ti=2

t0=newton2(y,ti,tol,maxiter)
x0=x(t0)
print("\nLe point d'impact est à {} mètres du tireur".format(x0))

#----------------ETUDE AVEC FROTTEMENTS----------------------------------

# Définition des équations horaires :

def vxf(t):
    vx=(M*V0*m.cos(a))/(k*V0*m.cos(a)*t+M)
    return vx

def vyf(t):
    arctgte=m.atan(m.sqrt(k/(M*g))*V0*m.sin(a))
    vy=m.sqrt(M*g/k)*m.tan(t*m.sqrt(k/(M*g))-arctgte)
```

```
    return vy

def xf(t):
    x=(M/k)*m.log((k*V0*m.cos(a)*t+M)/M)
    return x

def yf(t):
    arctgte=m.atan(V0*m.sin(a)*m.sqrt(k/(M*g)))
    A=m.cos(t*m.sqrt(g*k/M)-arctgte)
    B=M*g/(k*(V0*m.sin(a))**2 + M*g)
    y=h + (M/k)*m.log(A) - (M/(2*k))*m.log(B)
    return y

#---------------RESOLUTION y(t)=0------------------

# On utilise la méthode "newton2" pour ne pas avoir à calculer la dérivée

print("\n**********************************************************************")
print("Résolution AVEC prise en compte des forces de frottement dans l'air")
print("**********************************************************************")

from Newton import newton2

tol=0.000001; # A 1 microseconde près
maxiter=100; # 100 itérations au maximum
ti=2

tf0=newton2(yf,ti,tol,maxiter)
xf0=xf(tf0)
print("\nLe point d'impact est à {} mètres du tireur".format(xf0))



#--------------TRACES DES COURBES----------------------

import matplotlib.pyplot as plt

plt.close("all")

t=[]; xx=[]; yy=[];
tf=[]; xxf=[]; yyf=[];
for i in range(101):
    t.append(t0*i/100)
    xx.append(x(t[i]))
    yy.append(y(t[i]))
    tf.append(tf0*i/100)
    xxf.append(xf(t[i]))
    yyf.append(yf(t[i]))
figure1 = plt.figure()
plt.title("Trajectoire balle de tennis")
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
p1=plt.plot(xx,yy,color='b')
p2=plt.plot(xxf,yyf,color='r')
plt.axis('equal')
plt.show()
```